

IMPERIUNS BOT

Painel de Controle & Selfbot de Filas Free Fire (Discord)

Documento 1 de 2 — Roadmap Geral

Versão 1.0 · Abril/2026

Este roadmap descreve, em alto nível, todas as fases necessárias para reconstruir o bot do zero — desde a infraestrutura e o painel web até o motor de filas, detecção de partidas, envio de mensagens e operação contínua. O segundo documento aprofunda cada tópico apresentado aqui.

Visão Geral

O Imperiuns Bot é um sistema dividido em duas grandes peças: um **painel web** que o operador usa para configurar tokens, modos, orgs, mensagens, delays e estatísticas; e um **worker (selfbot)** que se conecta ao Discord via gateway com um ou mais tokens, entra automaticamente nas filas dos servidores parceiros, detecta quando uma partida é criada e envia mensagens/imagens dentro do canal recém-aberto.

O comportamento desejado, diferente de simplesmente preferir filas com ou sem players, é **revezar entre todos os modos e todas as orgs** até atingir o limite individual de filas que cada org permite por usuário. Em outras palavras: o bot precisa pulverizar a presença, garantindo que sempre haja o máximo possível de filas ativas simultâneas, sem concentrar em um único modo ou em uma única org.

Arquitetura em camadas

Camada	Responsabilidade
Painel Web (Frontend)	UI de configuração, dashboard de stats, logs ao vivo, controle Start/Stop.
API/Backend	Persistência de configuração, autenticação, broadcast de stats e logs (WebSocket).
Worker (Selfbot Engine)	Gateway Discord, descoberta de canais, entrada em filas, detecção de match, envio de
Camada de Dados	Tokens, configurações, métricas, log histórico, estado por instância (BOT1/BOT2).

Fases do Projeto

FASE 1 Discovery & Especificação	
Objetivo	Consolidar todas as regras de negócio: orgs, modos, formatos de embed, tipos de botão, padrões de canal de partida, limites por org, prioridades de mensagem.
Entregáveis	• Mapa de orgs (nome, guild_id, categorias, canais por modo) • Catálogo de variantes de embed e botões (Entrar/Sair, Gel Normal/Inf, Jogar Normal/Full UMP & XM8, etc.) • Glossário de regex/heurísticas para nomes de canal de match (fila-XXXX, partida-X) • Lista de modos e formatos suportados (1x1, 2x2, 3x3, 4x4, e variações x1/x2/x3/x4)
Depende de	—

FASE 2 Infraestrutura & Banco de Dados	
Objetivo	Subir base do projeto, banco relacional, fila/cache, schema de configuração, métricas e logs.
Entregáveis	• Schema: users, instances, tokens, orgs, channels, queues, matches, messages, stats, logs • Migrations e seeders iniciais (orgs e modos conhecidos) • Camada de criptografia para tokens em repouso • Estrutura de diretórios e padrões de código
Depende de	Fase 1

FASE 3 Backend / API & Auth	
Objetivo	Construir a API que serve o painel: autenticação, CRUD de configuração, broadcast em tempo real.
Entregáveis	• Login do operador (sessão segura) • Endpoints REST de configuração (tokens, orgs, modos, mensagens, delay, rotação) • WebSocket para stats e logs em tempo real • Endpoints de controle: start/stop por instância, reset de stats
Depende de	Fase 2

FASE 4	Painel Web (Frontend)
Objetivo	Reproduzir o painel mostrado nas referências: status, instâncias BOT1/BOT2, controles, stats, configuração e logs.
Entregáveis	- Tela de Controle com botão Start/Stop e seletor de instância - Cards de estatísticas (Entradas, Na Fila, Partidas, DMs, Uptime) - Formulário de Configuração (tokens, rotação, categoria, orgs, delay, modos, mensagens) - Console de Logs ao vivo com níveis e filtro - Tema escuro com identidade Imperiuns
Depende de	Fase 3

FASE 5	Worker — Núcleo do Selfbot
Objetivo	Conectar tokens ao gateway do Discord, manter sessões resilientes e expor API interna para os módulos superiores.
Entregáveis	- Gerenciador de sessões multi-token com reconexão e backoff - Rotação automática de tokens a cada N minutos - Listener de eventos relevantes (READY, GUILD_CREATE, MESSAGE_CREATE, CHANNEL_CREATE, GUILD_MEMBER_ADD) - Camada anti-rate-limit (bucket por rota, jitter, fila de envio)
Depende de	Fase 2

FASE 6	Descoberta de Orgs, Categorias e Canais
Objetivo	Mapear dinamicamente em quais guilds o token está e quais canais correspondem aos modos configurados.
Entregáveis	- Filtro por categoria (Mobile, Misto, Emulador) e por nomes de canal (1x1-mob, 2x2-mob, etc.) - Cache por guild com TTL e invalidação em eventos de canal - Whitelist/blacklist de orgs no painel - Resolução de prioridade por org (nome ou guild_id)
Depende de	Fase 5

FASE 7	Parser de Embeds & Botões
Objetivo	Interpretar os cards de fila de cada org, mesmo com visual diferente, e mapear para uma estrutura comum.

Entregáveis	• Adaptadores por org/template (Moreira, Colombia, Gold, Bounty, etc.) • Modelo unificado: {modo, formato, valor, jogadores, variantes_de_botao} • Reconhecimento de botões (Entrar, Sair, Gel Normal, Gel Inf, Jogar Normal, Jogar Full UMP & XM8) • Resiliência a edição de mensagens e atualização de embeds
Depende de	Fase 6

FASE 8	Motor de Filas (Round-Robin Org × Modo)
Objetivo	Decidir, em ciclo, em qual fila o bot entra a seguir, respeitando limites e modos permitidos.
Entregáveis	• Algoritmo round-robin justo entre orgs e modos • Controle do limite máximo de filas simultâneas por org/usuário • Suporte a entrar tanto em filas com player quanto vazias, sem preferência • Delay configurável entre cliques e jitter humano
Depende de	Fase 7

FASE 9	Deteção de Partida (Match)
Objetivo	Identificar com segurança quando um canal de partida foi criado para o bot.
Entregáveis	• Listener de CHANNEL_CREATE / MEMBER_ADD em categorias de partida • Regex/heurística para nomes (fila-XXXXXX, partida-N, SUA PARTIDA N) • Associação match ↔ fila de origem ↔ adversário (mention/ID) • Deduplicação para evitar processar a mesma partida duas vezes
Depende de	Fase 8

FASE 10	Mensagens dentro da Partida
Objetivo	Enviar a mensagem/imagem certa, com as variáveis substituídas e respeitando regras por org.
Entregáveis	• Substituição de variáveis: {adversary_mention}, {adversary_id}, {channel_name} • Mensagem principal global + mensagem secundária por org (prioridade por nome ou guild_id) • Suporte a imagem anexa configurada no painel • Controle de envio único por partida com idempotência
Depende de	Fase 9

FASE 11	Telemetria, Stats & Logs
Objetivo	Atualizar painel em tempo real com contadores e logs estruturados.
Entregáveis	• Contadores: Entradas em fila, Filas ativas, Partidas, DMs, Uptime • Reset de stats por instância • Log estruturado por nível (INFO/WARN/ERROR) com origem (org, canal, ação) • Persistência de log com rotação
Depende de	Fase 4 + 5

FASE 12	Hardening, Observabilidade & Operação
Objetivo	Tornar o sistema robusto para uso contínuo e múltiplas instâncias.
Entregáveis	• Tratamento de 401/403 (token inválido) e 429 (rate-limit) com isolamento por token • Health-checks e auto-restart de sessão • Backups de configuração e exportação/importação • Documentação de operação e troubleshooting
Depende de	Todas as anteriores

Linha do Tempo Sugerida

A linha do tempo abaixo é uma sugestão de ordem prática de execução, agrupando fases que podem caminhar em paralelo. Não representa prazos fechados — são blocos lógicos de trabalho.

Bloco	Fases	Descrição
Bloco A — Fundação	1, 2	Especificação completa, banco e infraestrutura mínima.
Bloco B — Painei	3, 4	Backend de configuração e frontend do painel utilizável.
Bloco C — Núcleo	5, 6, 7	Selfbot conectando, descobrindo canais e lendo embeds.
Bloco D — Operação	8, 9, 10	Entrar em filas, detectar partidas, enviar mensagens.
Bloco E — Produção	11, 12	Telemetria refinada e endurecimento para uso real.

Critérios de “pronto”

- Operador consegue logar no painel, colar tokens, escolher orgs/modos/mensagens e salvar.
- Ao apertar Start, o bot conecta todos os tokens e aparece como “Conectado / Rodando”.
- O bot revezando entre orgs e modos atinge o limite de filas simultâneas de cada org.
- Cada partida criada gera uma única mensagem dentro do canal, com as variáveis corretas.
- Stats e logs do painel batem com o comportamento real observado no Discord.
- Tokens inválidos, 429 e desconexões não derrubam o sistema — apenas o token afetado.

Riscos & Pontos de Atenção

Termos de Serviço do Discord

Selfbots violam o ToS do Discord. O operador assume o risco de banimento de conta. O painel deve manter o aviso já existente: “Use por sua conta e risco.”

Diferenças de embed por org

Cada org pode mudar visual, ordem dos botões e nomes (Entrar/Sair, Gel Normal/Inf, Jogar Normal/Full UMP & XM8). É necessário um sistema de adaptadores extensível.

Rate-limit e 429

Erros 429 já aparecem nos logs reais. O motor precisa isolar a punição por token e por rota, com backoff exponencial e jitter.

Limite por org

Cada org permite apenas N filas simultâneas. O algoritmo de round-robin precisa respeitar esse teto e liberar slots quando uma fila vira partida ou é cancelada.

Idempotência de mensagens

Sob reconexão ou eventos duplicados, é fácil mandar a mesma mensagem duas vezes na mesma partida. Precisa de chave única por (token, channel_id).

Rotação de tokens

A troca de tokens em ciclo precisa preservar o estado de filas ativas — ou desconectar limpo antes de trocar para evitar “fantasma” em uma org.